

Lecture 1

1. x4012	x0031	'1'
x4013	x0039	'9'
x4014	x0000	NUL

↑ Address ↑ Real Value

* Adding Addresses doesn't work. Adding strings needs more work.

We want to get string "19" ...

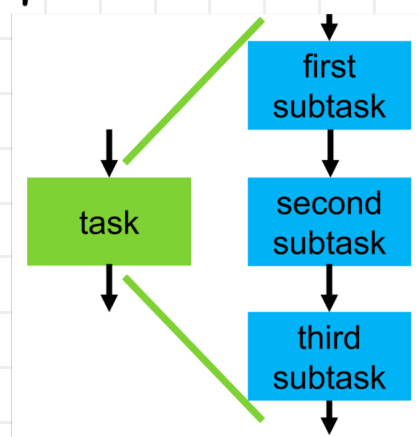
2. Review: Systematic Decomposition:

human language → break into simpler tasks repeatedly

"Simpler tasks" mean { correspond to statements in C
but the iterative construct is simpler.

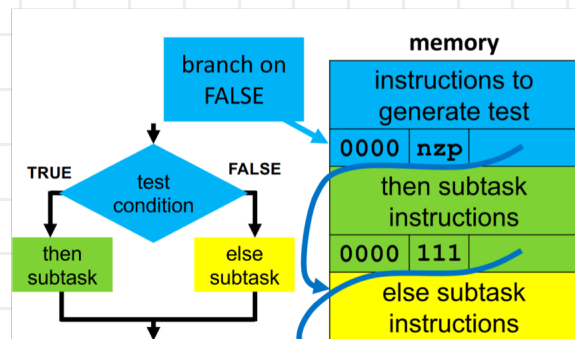
① Sequential Construct

memory: first
 second
 third



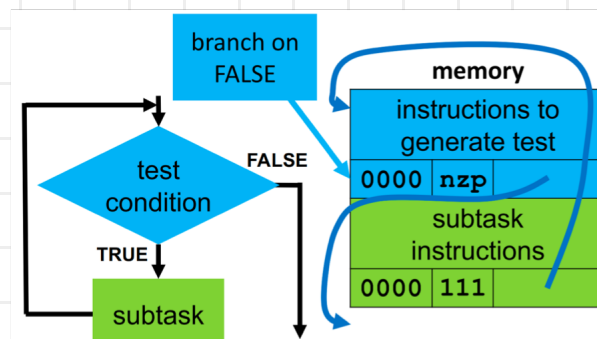
② Conditional Construct

memory: Use branch



③ Iterative Construct

memory: Use branch



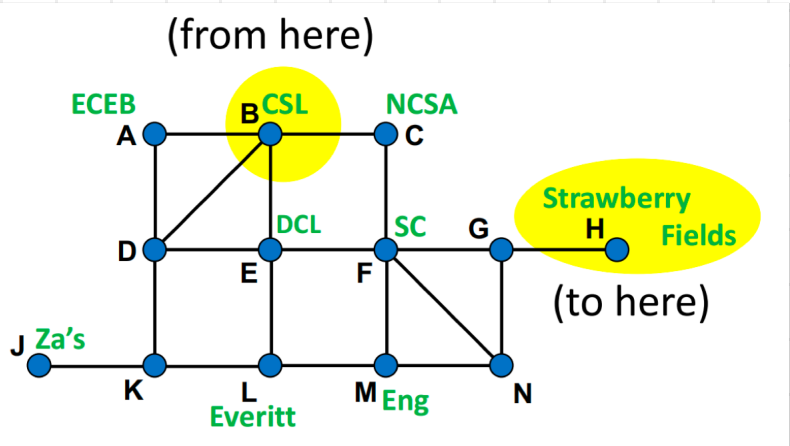
3. Review : Good Design { ① simpler (or feasible) approach
 ② easy to understand and test

e.g. Find Path

Idea 1: Find all path → repeated x

Idea 2: Find all simple path*

Idea 3: Make a queue of nodes and keep track of the best previous location for each code.

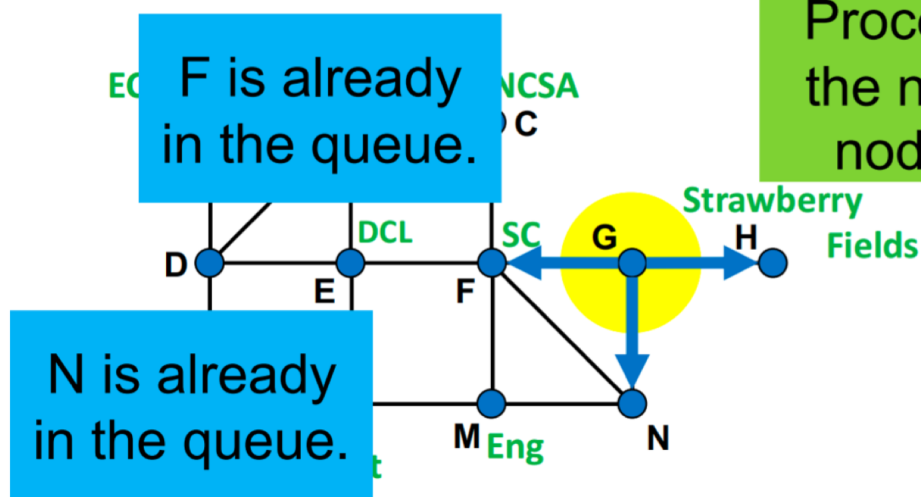


* "Simple path" in mathematics is one that includes any node at most one time.

So: check each node's neighbour:

distance 0/ 1/ 2/ 3/ 4/
 explored ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

queue	B	A	D	E	C	K	L	F	J	M	N	G	H
previous	-	B	B	B	B	D	E	E	K	L	F	F	G



G → H
 F → G ⇒ B E F G H
 E → F
 B → E

This approach is called breadth-first search (BFS), which explores nodes in order of distance

Lecture 2

1. Review:

- Five major components:
 1. Memory
 2. Input
 3. Output
 4. Processing unit
 5. Control unit

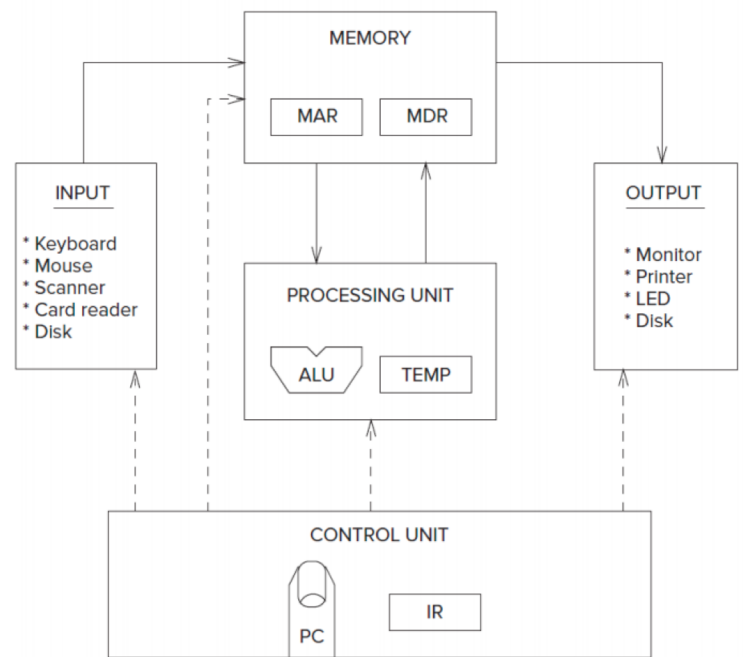


Figure 4.1 The von Neumann model, overall block diagram.

LC3 Instruction Set Architecture (ISA)

- ① Eight GPRs $R0 \sim R7$
- ② Data type: 16-bit 2's complement integers
- ③ Addressing: Locations $x0000 - xFFFF$
- ④ Addressing codes: immediate, register, PC-relative, base-offset, indirect

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD*	0001			DR				SR1	0	00					SR2	
ADD*	0001			DR				SR1	1						imm5	
AND*	0101			DR				SR1	0	00					SR2	
AND*	0101			DR				SR1	1						imm5	
BR	0000			n		z		p								PCOffset9
JMP	1100			000				BaseR								000000
JSR	0100			1												PCOffset11
JSRR	0100			0		00		BaseR								000000
LD*	0010			DR												PCOffset9
LDI*	1010			DR												PCOffset9
LDR*	0110			DR				BaseR								offset6
LEA	1110			DR												PCOffset9
NOT*	1001			DR				SR								111111
RET	1100			000				111								000000
RTI	1000															000000000000
ST	0011			SR												PCOffset9
STI	1011			SR												PCOffset9
STR	0111			SR				BaseR								offset6
TRAP	1111			0000												trapvect8
reserved	1101															

Figure A.2 - P&P 3rd Ed.

Operation

Data movement

Program flow

Addressing modes: LC-3 supports 5 modes, x86 has 20+

- immediate (or literal): $ADD\ R1, R4, \#-2$
- register addressing mode: $NOT\ R3\ R5$

LD: 把 LABEL 地址里的取出

LDI: 把 LABEL 里的值作地址再取一次

LDR: 把 R2 的值作基址, 偏移 # 再取值

LEA: 取 LABEL 本身地址

```
.ORIG x3000
x3000 LD R1, LABEL
x3001 LDI R2, LABEL
x3002 LDR R3, R2, #1
x3003 LEA R4, LABEL
x3004 LABEL .FILL x4001
END
```

Assume

Address	Data
; x4001	x6001
;	
; x6001	x7001
; x6002	x7002

Exercise

What are the values of R1, R2, R3 & R4 at each step?

R1: x4001

R2: x6001

R3: x7002

R4: x3004

2. Memory-Mapped I/O 内存映射 I/O

(1) Assign a memory address to each device register:

xFE00 - xFFFF are reserved (Reserving Address) and allocated to I/O device registers

- Use data movement instruction (load/store) for control and data transfer
- LC-3 Input and Output Device Registers

KBDR - store **ASCII value** of character entered from **keyboard**

KBSR - let processor know a new value is entered

DDR - store **ASCII value** of character to be displayed on **monitor**

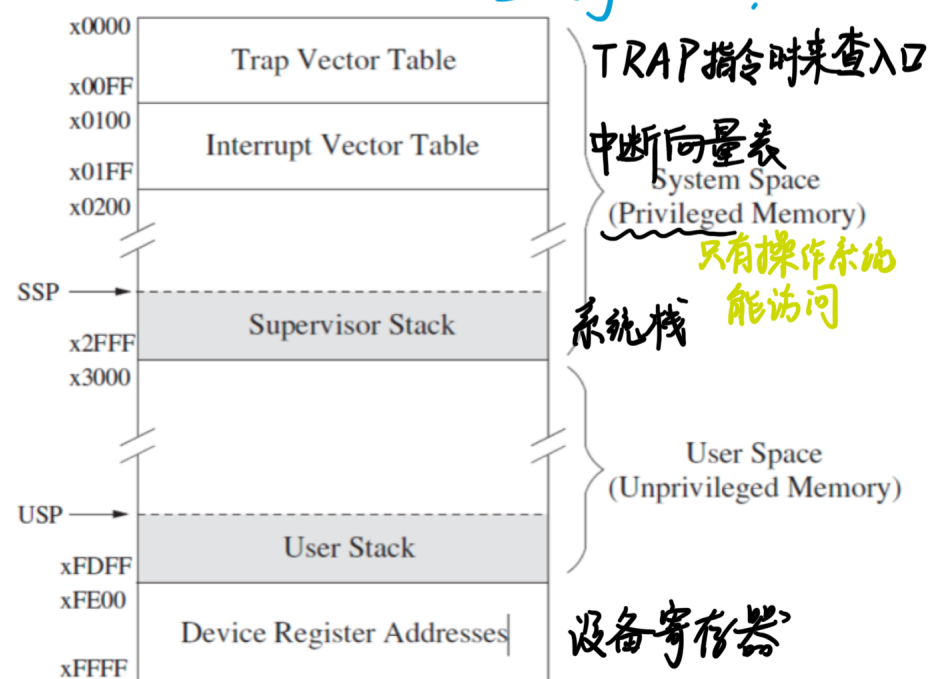
DSR - let processor know a new value is ready to be displayed

Address	I/O Register Name	Comments
x0000		; system space
...		
x3000		; user space
		; programs
		; and data
...		
xFE00	KBSR	; Device registers maps
xFE02	KBDR	
xFE04	DSR	
xFE06	DDR	
...		
xFFFF		

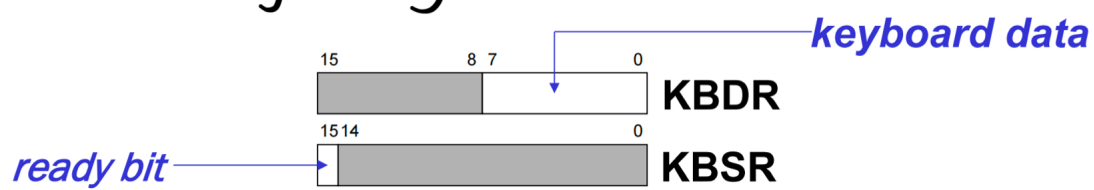
These are the memory addresses to which the device registers (KBDR, etc.) are **mapped**

The device registers physically are **separate circuits** from the memory, **where are they?**

In keyboard !



(2) handshaking using KBDR and KBSR:



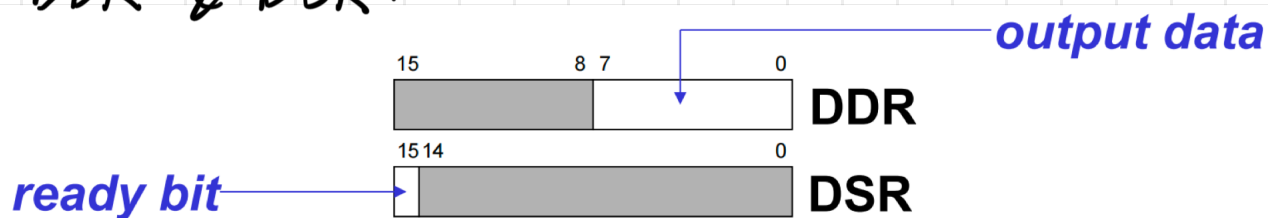
⇒ When a char is typed by user in the keyboard,

- Its ASCII code is played in KBDR [0:7]
- KBSR [15] is set to 1
- Keyboard is disabled.

⇒ When KBDR is read by CPU

- KBSR [15] is set to 0
- Keyboard is enabled.

(3) DDR & DSR:



⇒ When monitor is ready to display another char

- DSR [15] is set to 1

⇒ When new char is written to DDR

- DSR [15] is set to 0
- Any other chars written to DDR are ignored
- DDR [7:0] is displayed.

* Poiling 轮询: 键盘侧 + CPU侧

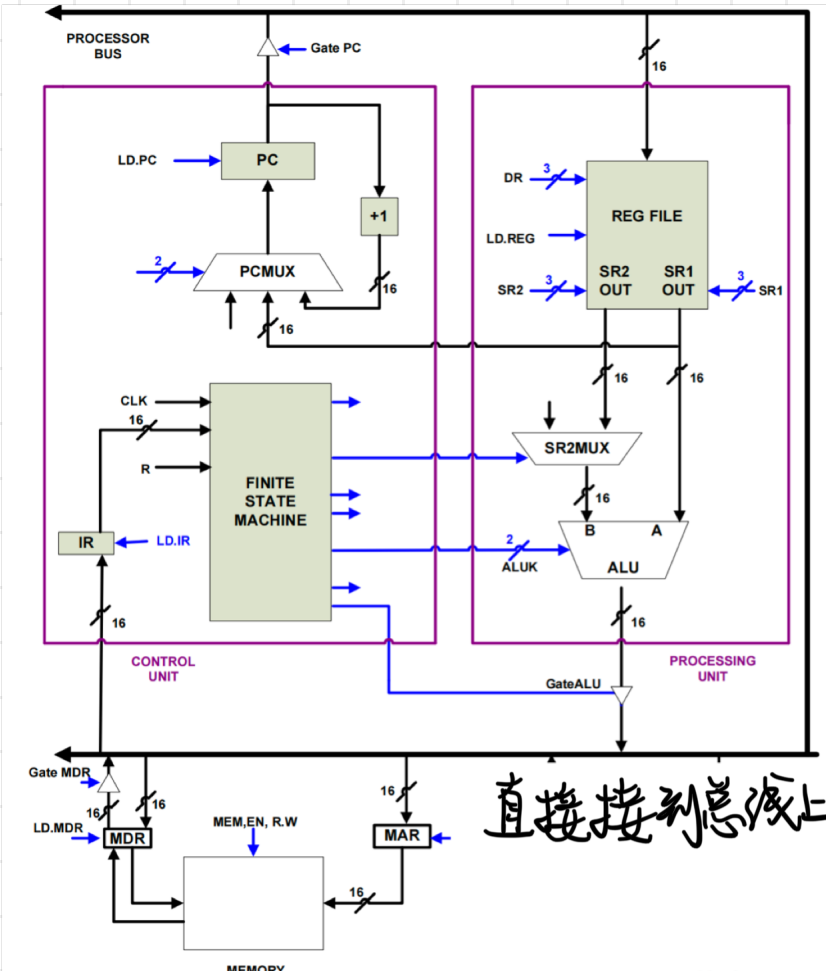
CHECK LDI R0, KBSR

BRzp CHECK

第15位是0, 原地等待

LDI R0, KBDR

第15位是1, 读数据.



直接接到总线上

